

DF 4조 최종 발표

Book Score Prediction

1. 회귀 데이터 셋 소개

회귀

회귀 데이터셋

ID : 샘플 고유 ID

User-ID : 유저 고유 ID

Book-ID : 도서 고유 ID

Age : 나이

Location : 지역

Book-Title : 도서 명

Book-Author : 도서 저자

Year-Of-Publication : 도서 출판 년도 (-1일 경우 결측 혹은 알 수 없음)

Publisher : 출판사

Book-Rating : 유저가 도서에 부여한 평점 (0점 ~ 10점)

> 단, 0점인 경우에는 유저가 해당 도서에 관심이 없고 관련이 없는 경우



주제: 도서 추천 알고리즘 **AI** 모델 개발

=> 유저 정보와 도서 정보를 바탕으로, 유저가 부여한 도서 평점을
회귀 예측하는 **AI** 모델을 개발하는 것이 목표

데이터 타입, 기초통계량, 결측값 확인

```

▶ print(df.info())
print(df.describe())

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 871393 entries, 0 to 871392
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                    871393 non-null object
1   User-ID              871393 non-null object
2   Book-ID              871393 non-null object
3   Book-Rating          871393 non-null int64
4   Age                  871393 non-null float64
5   Location              871393 non-null object
6   Book-Title           871393 non-null object
7   Book-Author          871393 non-null object
8   Year-Of-Publication  871393 non-null float64
9   Publisher            871393 non-null object
dtypes: float64(2), int64(1), object(7)
memory usage: 66.5+ MB
None

```

	Book-Rating	Age	Year-Of-Publication
count	871393.000000	871393.000000	871393.000000
mean	2.820579	36.799861	1968.903339
std	3.846906	12.065509	228.077752
min	0.000000	0.000000	-1.000000
25%	0.000000	31.000000	1991.000000
50%	0.000000	35.000000	1997.000000
75%	7.000000	41.000000	2001.000000
max	10.000000	244.000000	2021.000000

```

▶ print(df.isnull().sum())

... ID 0
User-ID 0
Book-ID 0
Book-Rating 0
Age 0
Location 0
Book-Title 0
Book-Author 0
Year-Of-Publication 0
Publisher 0
dtype: int64

```

```

[Duplicated rows] 0
[ID unique / total] 871393 / 871393
[ID duplicated] 0
[(User-ID, Book-ID) duplicated] 0

```

텍스트 컬럼 정규화

```
NA_TOKENS = {"", "n/a", "na", "none", "null", "unknown"}

def normalize_text_min(x):

    if pd.isna(x):
        return np.nan
    s = str(x).strip().lower()
    s = re.sub(r"Ws+", " ", s)
    if s in NA_TOKENS:
        return np.nan
    return s
```

◀ 문자열로 들어온 결측 표현들까지 결측으로 간주하기 위한 토큰 (NA_TOKENS)

```
for col in ["Location", "Book-Title", "Book-Author", "Publisher"]:
    df[col] = df[col].apply(normalize_text_min)
```

- 텍스트 정규화:
 - 앞뒤 공백 제거 (strip)
 - 소문자 통일 (lower)
 - 연속 공백(2칸 이상)을 1칸으로 축약 (re.sub)
 - 결측 토큰이면 np.nan 반환 (결측 데이터를 NaN으로 통일되어 있지 않아 통일시키는 과정)

텍스트 컬럼 결측 행 삭제 - Book-Author, Publisher

- 작가/출판사 결측 행 삭제

```
before = len(df)
df = df.dropna(subset=["Book-Author", "Publisher"]).copy() # 특정 컬럼 NaN인 행 삭제
print(f"\n[Drop rows where Book-Author or Publisher missing] {before} -> {len(df)}")
```

```
[Drop rows where Book-Author or Publisher missing] 871393 -> 871285
```

텍스트 컬럼 결측 행 삭제 - Location

```
df["Location_country"] = (
    df["Location"].astype(str)
    .str.strip().str.lower()
    .str.replace(r"Ws+", " ", regex=True)
    .str.rsplit(",", n=1).str[-1] # 마지막 쉼표 기준 오른쪽만 추출
    .str.strip()
)
```

```
df.loc[df["Location_country"].isin(NA_TOKENS), "Location_country"] = np.nan
before = len(df)
df = df.dropna(subset=["Location_country"]).copy() # [web:284]
print(f"[Drop rows where Location_country missing] {before} -> {len(df)}")
```

```
[Drop rows where Location_country missing] 871285 -> 839098
```

```
df_final = df_final.drop(columns=["Location"])
```

◀ Location_country 생성 (마지막 쉼표 뒤만 추출)
 > Location 컬럼이 도시, 주, 나라 형식으로 되어 있어
 나라만 추출하여 데이터 전처리 수행

◀ 결측 토큰은 NaN으로 통일하고 행 삭제

◀ 기존 Location 컬럼은 삭제

텍스트 컬럼 결측 행 삭제 - Year-Of-Publication

- Year-Of-Publication 컬럼에서 -1일 경우 결측 혹은 알 수 없음이므로 결측 토큰으로 간주하고 행 삭제

```
df["Year-Of-Publication"] = pd.to_numeric(df["Year-Of-Publication"], errors="coerce")
df.loc[df["Year-Of-Publication"] == -1, "Year-Of-Publication"] = np.nan
print("\n[YOP NA count after -1->NaN]:", df["Year-Of-Publication"].isna().sum())

before = len(df)
df = df.dropna(subset=["Year-Of-Publication"]).copy()
print(f"[Drop rows where Year-Of-Publication missing] {before} -> {len(df)}")

print("\nDone (preprocessing before rating filtering / p-core). Current df shape:", df.shape)
```

[YOP NA count after -1->NaN]: 11039

[Drop rows where Year-Of-Publication missing] 839098 -> 828059

Done (preprocessing before rating filtering / p-core). Current df shape: (828059, 11)

중간 점검 - 결측치 삭제 후 국가 상위 빈도 확인

- 이상한 값이 존재하지 않음을 확인

```
print("\nTop 30 Location_country:")  
print(df["Location_country"].value_counts().head(30))
```

```
Top 30 Location_country:  
Location_country  
usa                626212  
canada             77623  
united kingdom    25102  
germany           22531  
australia         14425  
spain             12291  
france            8165  
portugal          5846  
malaysia          4496  
new zealand       4464  
netherlands       3904  
switzerland       3228  
italy             2449  
austria           2195  
iran              1255  
romania           1046  
finland           1029  
singapore         1025  
dominican republic 902  
brazil            745  
sweden            690
```

```
ireland           663  
philippines       520  
japan             510  
belgium           417  
china             410  
norway            357  
us                291  
denmark           287  
hong kong         274  
Name: count, dtype: int64
```


Book-Rating(도서 평점)에서 0 제거 후 1~10만 회귀 대상으로 사용

- 원 데이터는 0점(implicit, 유저가 도서에 부여한 평점이 아닌 관측되지 않은 값)이 다수(60% 이상)를 차지하는데 0점을 평점처럼 포함하여 바로 회귀를 수행하면 예측이 0에 치우치는 등 결과가 왜곡될 수 있어 1~10점(explicit)만 남기는 것을 선택

```
df_explicit = df[df["Book-Rating"].between(1, 10)].copy()
print("\n[df_explicit] shape:", df_explicit.shape)
print("\n[df_explicit] rating distribution:\n", df_explicit["Book-Rating"].value_counts().sort_index())

user_cnt_pre = df_explicit.groupby("User-ID").size()
item_cnt_pre = df_explicit.groupby("Book-ID").size()
print("\n[Pre p-core] user rating count describe:\n", user_cnt_pre.describe())
print("\n[Pre p-core] book rating count describe:\n", item_cnt_pre.describe())

...
[df_explicit] shape: (302121, 11)
[df_explicit] rating distribution:
Book-Rating
1      1156
2      1833
3      4007
4      5932
5     35999
6     24819
7     52208
8     71974
9     47589
10    56604
Name: count, dtype: int64
```

```
[Pre p-core] user rating count describe:
count      58538.000000
mean         5.161109
std         26.015358
min          1.000000
25%          1.000000
50%          1.000000
75%          3.000000
max         3408.000000
dtype: float64

[Pre p-core] book rating count describe:
count     125608.000000
mean         2.405269
std          6.826811
min          1.000000
25%          1.000000
50%          1.000000
75%          2.000000
max          575.000000
dtype: float64
```

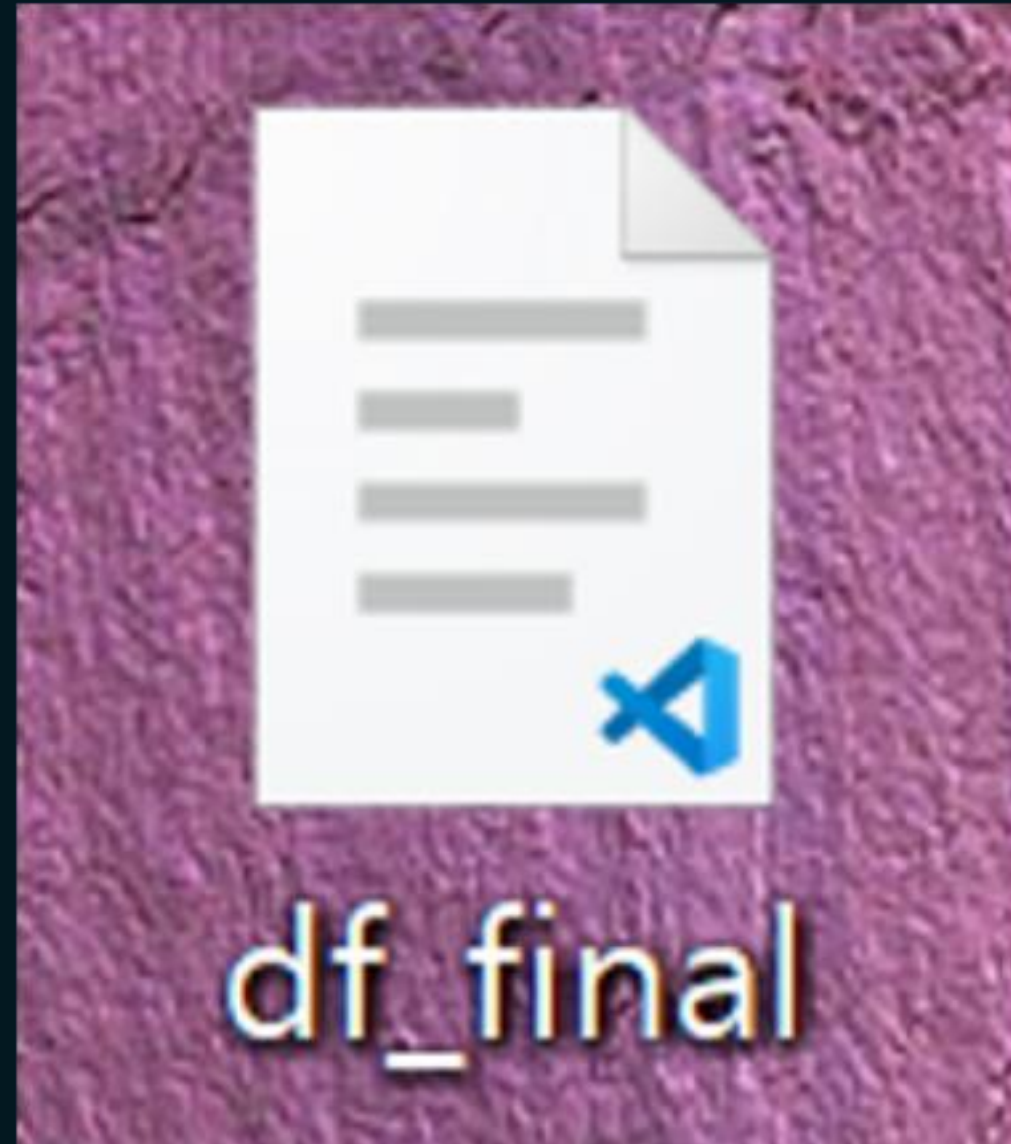
k-core 필터링 (min_user=3, min_item=3)

- 유저, 아이템 당 평점 수가 너무 적으면(콜드스타트, 희소성) 추천 모델 학습이 불안정해질 수 있어 유저와 아이템 당 평점 수가 각각 최소 3회 이상인 데이터만 남기도록 (3,3) k-core 필터링을 적용함.

```
def k_core_filter(df_in, user_col="User-ID", item_col="Book-ID", min_user=3, min_item=3, max_iter=10):  
    out = df_in.copy()  
    for _ in range(max_iter):  
        before = len(out)  
  
        # 유저 기준: min_user 미만 제거  
        user_cnt = out[user_col].value_counts()  
        out = out[out[user_col].isin(user_cnt[user_cnt >= min_user].index)]  
  
        # 아이템 기준: min_item 미만 제거  
        item_cnt = out[item_col].value_counts()  
        out = out[out[item_col].isin(item_cnt[item_cnt >= min_item].index)]  
  
        if len(out) == before:  
            break  
    return out  
  
df_final = k_core_filter(df_explicit, min_user=3, min_item=3)  
print("\n[df_final after k-core (3,3)] shape:", df_final.shape)  
  
print("min ratings per user:", df_final.groupby("User-ID").size().min())  
print("min ratings per book:", df_final.groupby("Book-ID").size().min())
```

```
[df_final after k-core (3,3)] shape: (132600, 11)  
min ratings per user: 3  
min ratings per book: 3
```


df_final 저장



```
df_final.to_csv("df_final.csv", index=False)
```

 df_final.csv

텍스트 -> TF-IDF -> TruncatedSVD -> 특성

- 회귀 모델(Ridge)는 텍스트 문자열을 그대로 학습 불가하기 때문에 텍스트를 숫자로 변환.
- TF-IDF: 문서에서 자주 나오지만 전체 문서에서는 흔하지 않은 단어에 더 높은 점수를 주는 방식. (결과로 보통 단어 수만큼 차원이 생김)
- Truncated SVD: TF-IDF로 만든 고차원 희소 벡터를 정보를 최대한 유지하면서 더 작은 차원으로 압축
- 특성: 최종적으로 Ridge에 들어가는 입력 숫자들

```
def text_pipe_safe(n_components, ngram_range=(1,2), min_df=2, max_df=0.9):  
    return Pipeline([  
        ("imputer", SimpleImputer(strategy="constant", fill_value="")),  
        ("to_1d", FunctionTransformer(lambda x: x.ravel(), validate=False)),  
        ("tfidf", TfidfVectorizer(  
            ngram_range=ngram_range,  
            max_features=MAX_FEATURES,  
            min_df=min_df,  
            max_df=max_df  
        )),  
        ("svd", TruncatedSVD(n_components=n_components, random_state=42)),  
    ])
```

```
(  
    "title",      text_pipe_safe(TITLE_NCOMP, (1,3)), ["Book-Title"]),  
    ("author",    text_pipe_safe(50, (1,1)), ["Book-Author"]),  
    ("publisher", text_pipe_safe(30, (1,1)), ["Publisher"]),  
    ("loc",       text_pipe_safe(10, (1,1)), ["Location_country"]),  
)
```


Target Encoding: 고차원 범주형 변수 처리

- User-ID와 Book-ID는 수만 개의 고유값으로 One-Hot Encoding이 비효율적(차원이 너무 커짐)
- 따라서 Target Encoding을 적용 (Target Encoding: 각 범주를 해당 범주의 평균 평점으로 변환)
- cv=5: 5-fold 교차 검증

```
user_te_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("te", TargetEncoder(smooth="auto", cv=5)),
])
```

```
book_te_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("te", TargetEncoder(smooth="auto", cv=5)),
])
```

```
( "user_te", user_te_pipe, [user_col] ),
( "book_te", book_te_pipe, [book_col] ),
```

데이터 분할 : Warm-start 설정의 User-based Holdout Split

```
def user_item_holdout_split(df, user_col, test_size=0.2, min_ratings_per_user=3, random_state=42):  
    rng = np.random.RandomState(random_state)  
    train_idx, test_idx = [], []  
  
    for uid, g in df.groupby(user_col):  
        idx = g.index.to_numpy()  
        n = len(idx)  
  
        if n < min_ratings_per_user:  
            continue  
  
        n_test = int(np.floor(n * test_size))  
        n_test = max(1, n_test)  
        n_test = min(n_test, n - 1)  
  
        test_pick = rng.choice(idx, size=n_test, replace=False)  
        test_set = set(test_pick.tolist())  
  
        for i in idx.tolist():  
            (test_idx if i in test_set else train_idx).append(i)  
  
    return np.array(train_idx), np.array(test_idx)
```


데이터 분할 : Warm-start 설정의 User-based Holdout Split

- Warm-start: 이미 과거 상호작용(평점/클릭 등) 기록이 있는 유저/아이템에 대해 추천/예측을 수행하는 상황(=일부 이력이 Train에 존재)
- Cold-start: 새 유저/새 아이템처럼 과거 상호작용 데이터가 거의/전혀 없어, 개인화 추천이 어려운 상황
- User 단위로 분할하여 Train/Valid/Test에 동일 유저가 존재하도록 구성

```
# Step 1) 전체 -> train_valid / test
train_valid_idx, test_idx = user_item_holdout_split(df, user_col, 0.2, 3, 42)
df_train_valid = df.loc[train_valid_idx].copy()
df_test = df.loc[test_idx].copy()

# Step 2) train_valid -> train / valid
train_idx, valid_idx = user_item_holdout_split(df_train_valid, user_col, 0.2, 3, 43)
df_train = df_train_valid.loc[train_idx].copy()
df_valid = df_train_valid.loc[valid_idx].copy()
```

Pipeline 구조

- ColumnTransformer로 컬럼 타입별 전처리를 병렬 적용한 뒤 결과를 하나의 특성 행렬로 결합.
- 수치형: StandardScaler로 스케일 정규화.
- 최종적으로 Ridge Regression(alpha=10)으로 학습.
- L2 정규화로 고차원/상관 특성에서 과적합과 다중공선성 영향을 줄임.

```
num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

```
preprocess = ColumnTransformer(
    transformers=[

        ("user_te", user_te_pipe, [user_col]),
        ("book_te", book_te_pipe, [book_col]),

        ("title", text_pipe_safe(TITLE_NCOMP, (1,3)), ["Book-Title"]),
        ("author", text_pipe_safe(50, (1,1)), ["Book-Author"]),
        ("publisher", text_pipe_safe(30, (1,1)), ["Publisher"]),
        ("loc", text_pipe_safe(10, (1,1)), ["Location_country"]),

        ("num", num_pipe, num_cols),
    ],
    remainder="drop"
)
```

```
model = Pipeline([
    ("prep", preprocess),
    ("ridge", Ridge(alpha=RIDGE_ALPHA, random_state=42)),
])
```

결과

- Valid와 Test RMSE가 거의 유사하여 일반화 격차가 크지 않고 모델이 비교적 안정적임.
- 본 모델은 10점 만점 평점에서 평균 1.56점 오차를 보임

```
t0 = time.time()
model.fit(X_train, y_train)
pred_valid = model.predict(X_valid)

print("\n[VALID]")
print("RMSE:", root_mean_squared_error(y_valid, pred_valid))
print("MAE :", mean_absolute_error(y_valid, pred_valid))
print("R2  :", r2_score(y_valid, pred_valid))
print("valid fit+pred time:", f"{time.time()-t0:.1f}s")

model.fit(X_train_valid, y_train_valid)
pred_test = model.predict(X_test)

print("\n[TEST]")
print("RMSE:", root_mean_squared_error(y_test, pred_test))
print("MAE :", mean_absolute_error(y_test, pred_test))
print("R2  :", r2_score(y_test, pred_test))
```

```
[VALID]
RMSE: 1.56853215003281
MAE : 1.2139074886704024
R2  : 0.23418816866338066
valid fit+pred time: 192.6s
```

```
[TEST]
RMSE: 1.5586796932666305
MAE : 1.208977389210364
R2  : 0.24076525378378189
```


예측 샘플 분석

- 저평점(1~6)에서 MAE=2.17로 오차가 가장 크고, 중간(7~8)에서 MAE=0.58로 가장 안정적.
- 이는 저평점 예측이 상대적으로 어려움을 보여주며, 데이터 분포나 패턴 특성에 기인할 수 있음.

	y_true	y_pred
0	8	6.807901
1	9	7.418680
2	7	8.121734
3	9	8.196489
4	10	6.642903
5	9	8.445897
6	5	7.314800
7	10	8.930958
8	10	9.648162
9	7	7.184386

	count	mean
bin		
1~6	5451	2.169425
7~8	10566	0.583973
9~10	9649	1.351722

UserMean 베이스라인 대비 성능 개선

- UserMean 베이스라인: 각 유저의 Train 평균 평점을 그대로 예측값으로 사용하는 단순 기준 모델.
- 최종 모델은 UserMean 대비 TEST RMSE 0.0804 감소(4.90% 개선), MAE 0.0379 감소(3.03% 개선)를 달성.
- 유저 평균만으로는 설명 못 하는 변동을 책 메타/텍스트 특성까지 활용해 추가로 설명함.

```
from sklearn.metrics import mean_absolute_error, root_mean_squared_error
import numpy as np

target_col = "Book-Rating"
user_col = "User-ID"

# 최종 모델 TEST 성능(비교 대상)
FINAL_MODEL_TEST_RMSE = 1.5586796932666305
FINAL_MODEL_TEST_MAE = 1.208977389210364

# train에서만 유저 평균 계산
mu = df_train[target_col].mean()
user_mean_map = df_train.groupby(user_col)[target_col].mean()

# 유저 평균을 매핑
y_test = df_test[target_col].to_numpy()
pred_user = df_test[user_col].map(user_mean_map).to_numpy(dtype=float)
pred_user = np.where(np.isnan(pred_user), mu, pred_user)

# baseline 성능
user_rmse = root_mean_squared_error(y_test, pred_user)
user_mae = mean_absolute_error(y_test, pred_user)

# 최종 모델 대비 개선
rmse_abs_improve = user_rmse - FINAL_MODEL_TEST_RMSE
mae_abs_improve = user_mae - FINAL_MODEL_TEST_MAE
rmse_rel_improve = rmse_abs_improve / user_rmse * 100
mae_rel_improve = mae_abs_improve / user_mae * 100
```


감사합니다.